API SECURITY RISK ANALYSIS REPORT

Prepared By: Akshay Jain

Role: Cybersecurity Intern – Future Interns

Assessment Period: 25 March 2026 – 03 April 2026

1. Introduction

APIs (Application Programming Interfaces) are widely used to enable communication between applications and services. However, insecure APIs can expose sensitive data and increase the risk of cyberattacks.

This report presents a security assessment of selected public APIs to identify potential vulnerabilities and evaluate security controls.

2. Scope & Ethics

This assessment was conducted under strict ethical guidelines.

Scope includes:

- Public APIs only
- Safe testing methods
- Non-intrusive analysis

Excluded activities:

- Exploitation
- Denial-of-Service attacks
- Unauthorized access attempts

3. Tools Used

- Postman (API testing)
- Browser DevTools
- Public APIs (Reqres, JSONPlaceholder)

4. Methodology

The following methodology was used:

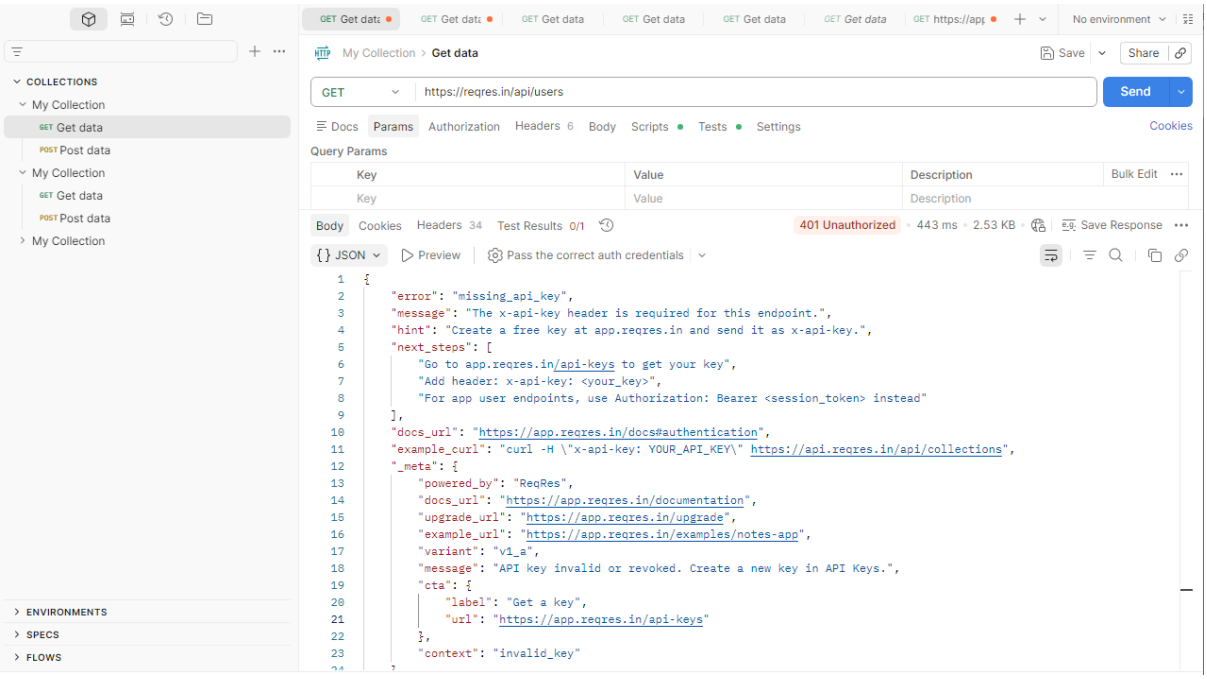
1. API endpoint identification
2. Request testing using Postman
3. Authentication analysis
4. Response evaluation
5. Security risk identification
6. Risk classification and documentation

5. APIs Tested

1. https://reqres.in/api/users
2. https://jsonplaceholder.typicode.com/posts

6. Findings

Finding 1 — Authentication Enforcement



Description:

The API requires an API key for access. Requests without authentication return a 401 Unauthorized response.

Evidence:

Response shows:

"error": "missing_api_key"

Impact:

This is a positive security control as it prevents unauthorized users from accessing API data.

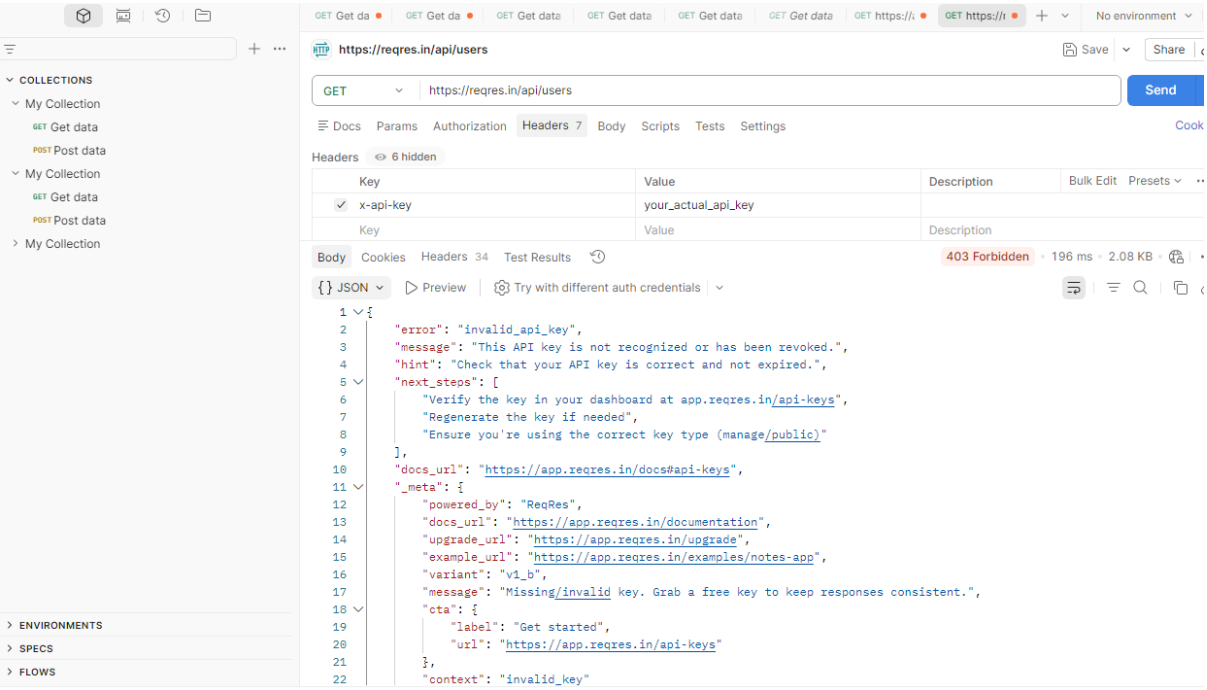
Risk Level:

Informational (Good Practice)

Recommendation:

Ensure API keys are securely stored and not exposed in client-side code.

Finding 2 — Invalid API Key Handling



Description:

When an invalid API key is provided, the API returns a 403 Forbidden response indicating access denial.

Evidence:

"error": "invalid_api_key"

Analysis

The API correctly enforces authentication and prevents unauthorized access. It also validates API keys and rejects invalid credentials, demonstrating proper access control implementation.

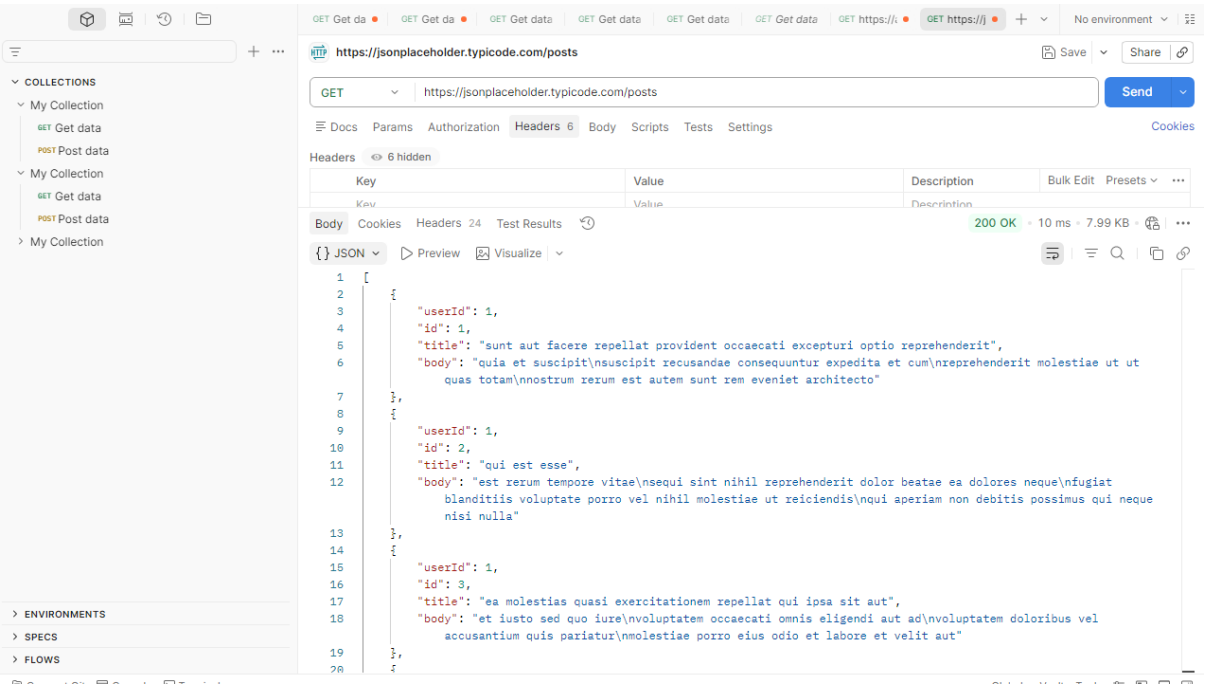
Risk Level

Informational (Secure Implementation)

Recommendation

- Continue enforcing API authentication
- Use secure API key storage
- Rotate API keys periodically

Finding 3 — No Authentication Required



Description:

The API endpoint allows access without any authentication.

Analysis

This indicates that the API does not enforce authentication controls to restrict access. While this may be acceptable for public or demo APIs, in real-world applications this behavior can lead to unauthorized access to sensitive data.

The absence of authentication increases the attack surface, as any external user or automated script can interact with the API without restrictions.

Impact

- Unauthorized users can access API data freely
- Sensitive information may be exposed if present
- Increased risk of data scraping and misuse
- Attackers can automate large-scale data extraction

Risk Level

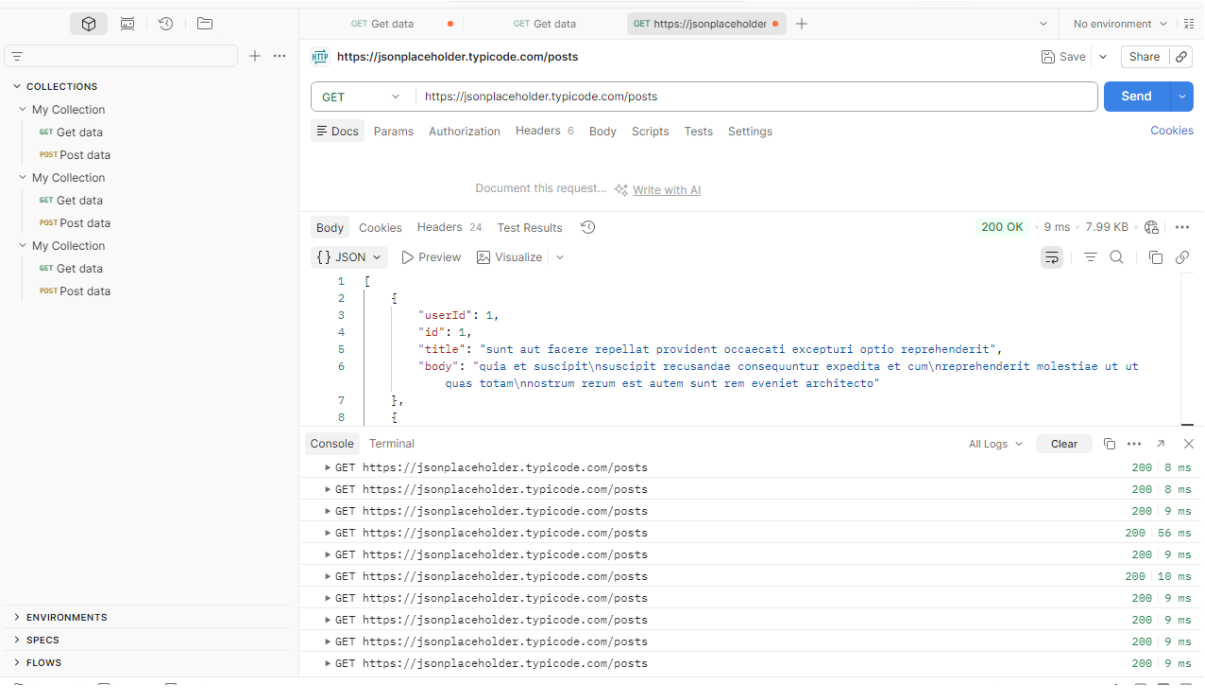
Medium

Reason:
There is no direct exploitation, but if sensitive data exists, the impact can be significant.

Remediation / Recommendation

- Implement proper authentication mechanisms such as:
    - API Keys
    - OAuth Tokens
    - JWT (JSON Web Tokens)
- Restrict access to authorized users only
- Apply role-based access control (RBAC) where necessary
- Ensure that only non-sensitive public data is exposed without authentication

Finding 4 — Lack of Rate Limiting



Description

The API does not implement rate limiting controls to restrict the number of requests from a single client within a specific time period.

Analysis

The API does not enforce any rate limiting or throttling mechanism. This allows unlimited requests from a single client within a short duration.

In real-world scenarios, this can lead to abuse such as automated attacks, excessive data extraction, or denial-of-service attempts.

Impact

- Attackers can perform **brute force attacks**
- API can be abused for **data scraping**
- Risk of **Denial-of-Service (DoS)** attacks
- Increased server load and performance degradation

Risk Level

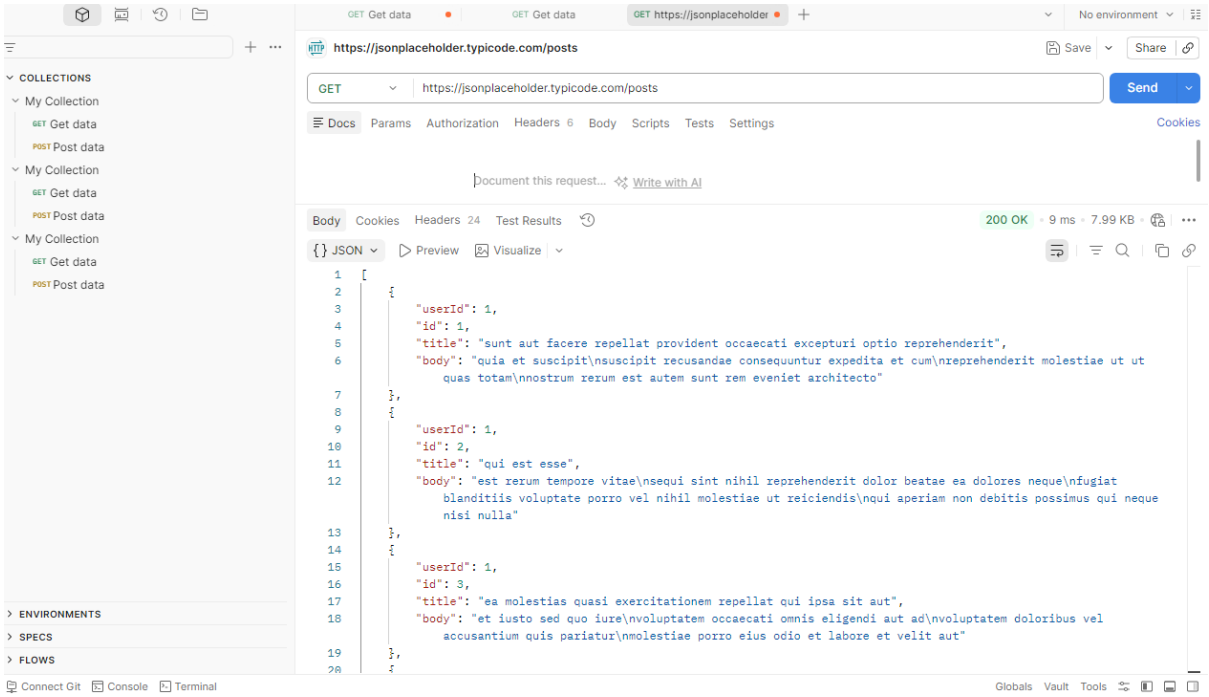
Medium

Reason:
No immediate breach, but enables large-scale abuse.

Remediation / Recommendation

- Implement rate limiting (e.g., limit requests per minute per IP)
- Use throttling mechanisms (e.g., 100 requests/minute)
- Apply API gateway protections (like AWS API Gateway / Nginx rate limiting)
- Monitor unusual traffic patterns

Finding 5 — Excessive Data Exposure



Description

The API returns more data than necessary in its response. Instead of limiting the response to only required fields, it exposes multiple attributes such as user identifiers and content data.

Evidence

The following API endpoint was tested:

https://jsonplaceholder.typicode.com/posts

The response includes fields such as:

userId, id, title, body

Analysis

The API exposes multiple data fields without any filtering or restriction. While this data may appear non-sensitive in a demo environment, in real-world applications similar behavior could lead to exposure of sensitive information such as:

- Personal user data
- Internal system identifiers
- Metadata not intended for public access

This indicates a lack of **data minimization**, where APIs should only return the minimum required information.

Impact

- Exposure of unnecessary data increases attack surface
- Attackers can gather structured data for further attacks
- May lead to **privacy violations** if sensitive data is exposed
- Enables **data scraping and profiling of users**

Risk Level

Medium

Reason:
Data is accessible and structured, which can be misused, even if not immediately critical.

Remediation / Recommendation

- Implement **data filtering** to return only required fields
- Use **response schemas** to control output
- Avoid exposing internal identifiers unless necessary

- Apply **role-based access control (RBAC)** for sensitive data
- Regularly review API responses for unnecessary data exposure

Finding 6 — SECURE ERROR HANDLING

Description

The API handles invalid requests appropriately by returning a generic error response without exposing internal system details.

Evidence

The following invalid endpoint was tested:

https://jsonplaceholder.typicode.com/invalid123

The API returned:

- Status Code: **404 Not Found**
- Response Body: {}

Analysis

The API provides a minimal and generic error response when an invalid endpoint is accessed. It does not reveal any internal system information such as stack traces, file paths, or server details.

This indicates proper implementation of secure error handling practices.

Impact

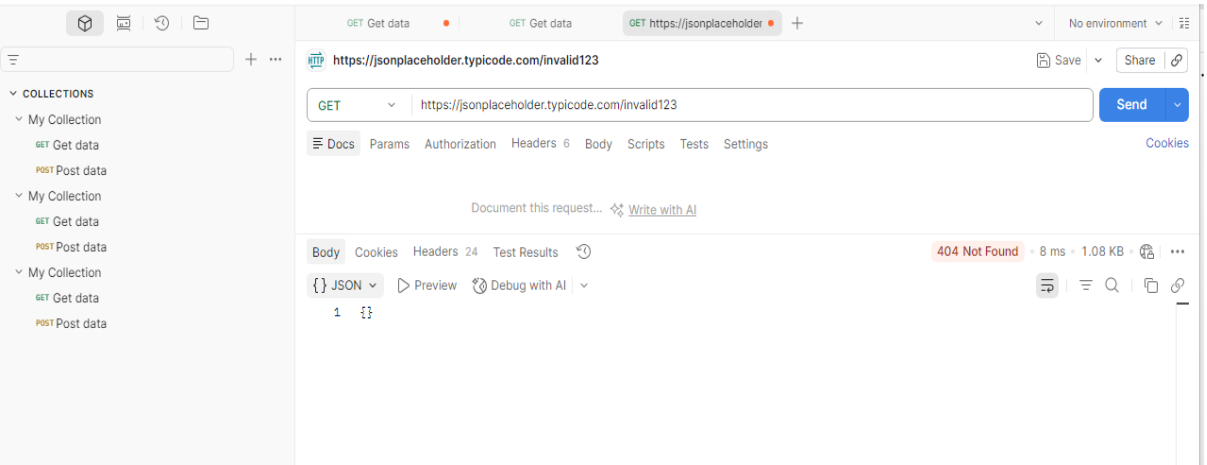
- Prevents attackers from gathering system information
- Reduces risk of endpoint enumeration
- Protects internal architecture from exposure

Risk Level

Informational (Secure Implementation)

Recommendation

- Continue using generic error messages
- Avoid exposing debugging information in production
- Log detailed errors internally for monitoring



Conclusion

The API security assessment identified both secure implementations and potential risks. While authentication and error handling are properly implemented in some APIs, other endpoints lack essential protections such as authentication and rate limiting.

By implementing the recommended controls, the overall security posture can be significantly improved.